



SCHOOL OF ENGINEERING SCIENCES
DEPARTMENT OF COMPUTER ENGINEERING
2ND SEMESTER, 2025/2026 ACADEMIC YEAR
COURSE: CPEN 208-SOFTWARE ENGINEERING
LECTURER: MR JOHN ASIAMAH
PROJECT 2

Table of Contents

1. Introduction.....	3
2. API Design.....	3
2.1 Approach.....	3
2.2 Endpoint List.....	3
3. Endpoint Details and Examples.....	4
3.1 Students.....	4
3.2 Registration and Login.....	4
3.3 Fees.....	5
3.4 Enrollment.....	5
3.5 Lecturer Assignments.....	5
4. Testing.....	5
5. What Was Submitted.....	5
6. Conclusion.....	5

1. Introduction

This project builds on Project 1. Project 1 set up a PostgreSQL database (CPEN208_PROJECT1) for the Computer Engineering Department, along with a Next.js web app for students to register, log in, and view a dashboard. This project, Project 2, takes the functions built in Project 1 and exposes them as a proper API/web service — a set of endpoints that any client (not just the Project 1 dashboard) could call to read or update the same data.

The API is a separate Next.js 14 project containing only API routes, no pages or user interface. It connects to the exact same CPEN208_PROJECT1 database used in Project 1, so all the real student, course, lecturer, and fee data carries over automatically.

2. API Design

2.1 Approach

Each of the five functionalities from Project 1 became one or more endpoints, following normal REST conventions: GET to read data, POST to create something, DELETE to remove something. All responses are JSON, and every endpoint returns a success field so a caller can easily tell whether a request worked.

2.2 Endpoint List

Method	Endpoint	Purpose
GET	/api/students	List every student
GET	/api/students/[id]	Get one student by ID
POST	/api/register	Register a new student, or claim an existing seeded record
POST	/api/login	Authenticate a student by email and password
GET	/api/fees	Outstanding fees for every student (calls <code>get_outstanding_fees</code>)
GET	/api/fees/[id]	Outstanding fees for one student
GET	/api/enrollment	List enrollments (all, or filtered by <code>?student_id=</code>)
POST	/api/enrollment	Enroll a student in a course
DELETE	/api/enrollment	Drop a student from a course
GET	/api/lecturer-courses	List lecturer-to-course assignments
GET	/api/lecturer-tas	List lecturer-to-TA assignments

3. Endpoint Details and Examples

3.1 Students

GET /api/students returns every student's basic details (never the password hash). GET /api/students/[id] returns just one student, or a 404 if the ID doesn't exist. Example:

```
GET /api/students/22325819

{
  "success": true,
  "student": {
    "student_id": "22325819",
    "first_name": "Nana Adwoa",
    "last_name": "Dansowaah Odoom",
    "email": "nanaadwoaodoom@gmail.com",
    "program": "Computer Engineering",
    "level": 200,
    "phone": "0505846691"
  }
}
```

3.2 Registration and Login

POST /api/register handles three cases: a fully registered student ID is rejected, a seeded but unclaimed student ID gets its email/phone/password filled in, and an unknown student ID creates a brand new record (also billed the standard fee automatically). POST /api/login checks the email and password against the stored bcrypt hash and returns the student's details on success.

```
POST /api/login
{ "email": "nanaadwoaodoom@gmail.com", "password": "*****" }

{
  "success": true,
  "student": {
    "student_id": "22325819",
    "first_name": "Nana Adwoa",
    "last_name": "Dansowaah Odoom",
    "email": "nanaadwoaodoom@gmail.com",
    "phone": "0505846691"
  }
}
```

3.3 Fees

GET /api/fees calls the get_outstanding_fees() function from Project 1 directly and returns the full JSON array it produces. GET /api/fees/[id] calls the same function and picks out one student.

```
GET /api/fees/22325819

{
  "success": true,
  "fees": {
    "student_id": "22325819",
    "first_name": "Nana Adwoa",
    "last_name": "Dansowaah Odoom",
    "total_billed": 5000.00,
    "total_paid": 3012.74,
    "outstanding_balance": 1987.26
  }
}
```

```
}  
}
```

3.4 Enrollment

GET /api/enrollment lists every enrollment, or just one student's if ?student_id= is added. POST enrolls a student in a course; DELETE drops them. Both take the same JSON body:

```
POST /api/enrollment  
{  
  "student_id": "22325819",  
  "course_id": "CPEN213",  
  "semester": "Semester 1, 2025/2026"  
}  
  
{ "success": true, "message": "Enrolled successfully." }
```

3.5 Lecturer Assignments

GET /api/lecturer-courses and GET /api/lecturer-tas return the full assignment lists, joined with lecturer, course, and TA names so the response is readable on its own rather than just a list of IDs.

4. Testing

Every GET endpoint was tested directly in the browser. POST and DELETE endpoints, which a browser can't send on its own, were tested using the Thunder Client extension in VS Code — sending a JSON body to each endpoint and checking the response, then confirming the change with a follow-up GET request.

5. What Was Submitted

Everything was pushed to a public GitHub repository, containing:

- The full Next.js 14 API source code (API routes only, no pages).
- The same database scripts from Project 1, in a database/ folder: create_tables.sql, functions.sql, and insert_data.sql — included here too since this API depends entirely on that database.
- The same full database backup (cpen208_backup.sql), since Project 2 uses the same live data as Project 1.

Repository: https://github.com/Claribel2010/cpen208_project2_api

6. Conclusion

This project takes every functionality built in Project 1 and exposes it as a working API: student information, fee payments (through the same outstanding-fees function), course enrollment, lecturer-to-course assignment, and lecturer-to-TA assignment. It runs against the same real class data as Project 1, and every endpoint was tested and confirmed working before submission.

The source code, database scripts, database backup, and this report are all available in the GitHub repository linked above.